

# T1

签到题。直接按位计算，可以求出每一个点作为根时的权值。

其中子任务 1 为暴力提供，子任务 2 简化为序列问题，子任务 3 提示拆位。

# T2

不是很难的题。观察到  $len \geq 5$  时，答案只在最高位  $p$  的个数为奇数的时候为  $2^p$ 。这部分的统计可以通过简单的分治实现。

$len \leq 4$  时需要枚举二叉树以得到答案。

子任务 1 为暴力提供，子任务 3 性质更为简单，子任务 2 为观察出性质后不会统计的选手提供。

# T3

根据传统的 dp 过程，维护  $cur$ 、 $ans$  和  $dif = cur - ans$ 。

每次执行  $cur = cur + a_i$ ， $dif = dif + a_i$ 。如果  $cur < 0$ ，则  $dif = cur$ ， $cur = 0$ 。如果  $dif > 0$ ，则  $dif = 0$ 。不需要显式地维护  $ans$ 。

考虑将所有数都表示成  $\sum c_i \cdot 2^i$  的形式，其中  $c_i = 0, 1, -1$ 。  
则所有加法和减法操作均摊只需要执行  $O(1)$  次。使用 set 维护这几个数即可。

使用 set 或者主席树直接维护 1 的连续段也是正确的。

部分分是通向正解的阶梯。

# T4

$n \leq 10$  时只需暴力枚举所有可能的树。

$k = 0$  时只需判断  $n$  是否形如  $2^k - 1$ 。

$k = 1$  时可以暴力枚举在满二叉树上不平衡的节点，是否合法只与此节点的深度有关。或者直接判断  $n$  是否形如  $2^p - 2^q - 1$ 。

对  $k$  较大的情形，考虑如下对树的刻画：从上往下考虑，假设当且节点高度为  $h$ ，则有两种情况

- 他有两个高度为  $h - 1$  的子树。
- 他有两个高度分别为  $h - 1$  和  $h - 2$  的子树。

按  $h$  从大到小处理，只需关心有多少已经生成的节点高度是  $h$  和  $h - 1$ ，在动态规划时记录有几个不平衡节点即可。直接实现此做法的时间复杂度约为  $O(n \log n \cdot k^3)$ 。

事实上，我们不关心具体的  $h$ ，因为假设有  $x$  个高度为 0 和  $y$  个高度为 -1 的点，总会生成出一棵总点数为  $2x + y - 1$  个点的树，因为每次合并两个点，生成一个点。假设有  $x$  个高度为  $h$  和  $y$  个高度为  $h - 1$  的点，有  $k$  个不平衡节点的方案数为  $f_{x,y,k}$ ，则转移时枚举  $x$  个种有  $c$  个不平衡的儿子，转移到状态  $f_{2x+y-c,c,c+k}$ 。

根据刚才的观察，我们只要求出所有  $2x + y - 1 = n$  的  $f_{x,y,k}$  的值。而上述转移时  $x$  几乎总是除 2。使用记忆化搜索倒过来转移，总共扫描到的状态数为  $O(\log n \cdot k^3)$ 。每次枚举转移时间复杂度即为  $O(\log n \cdot k^5)$ ，事实上常数很小，可以通过。